DATA STRUCTURES

Queue & Linked List

Theory + Easy C++ Program Examples

Concept • Operations • C++ Code • Output

QUEUE

Queue — Concept (FIFO)

Queue ek linear data structure hai jo FIFO (First In, First Out) principle par kaam karta hai — jo element sab se pehle add hota hai, wohi sab se pehle remove hota hai.

- **Enqueue —** Naya element rear (end) par add karna
- **Dequeue —** Element ko front se remove karna
- **Front / Peek —** Front element ko dekhna, bina remove kiye
- **isEmpty —** Check karna ke queue khaali hai ya nahi

Real-Life Example

Ticket counter par lagi line — jo pehle aata hai, usi ki ticket pehle banti hai.

Time Complexity

O(1)

for enqueue and dequeue

QUEUE

Queue — C++ Program

```cpp
#include <iostream>
using namespace std;
int q[5], front = -1, rear = -1;

void enqueue(int val) {
    if (rear == 4) { cout << "Full\n"; return; }
    if (front == -1) front = 0;
    q[++rear] = val;
}

void dequeue() {
    if (front > rear) { cout << "Empty\n"; return; }
    cout << "Removed: " << q[front++] << "\n";
}

int main() {
    enqueue(10); enqueue(20); enqueue(30);
    dequeue();
    dequeue();
    return 0;
}
```

Code Explanation

- q[5] fixed-size array queue hai
- enqueue() rear par value add karta hai
- dequeue() front se value remove karta hai
- front++ se front index aagay barhta hai

Output

```
Removed: 10
Removed: 20
```

LINKED LIST

Linked List — Concept

Linked List ek linear data structure hai jahan har element (node) apni data ke sath agle node ka address (pointer) bhi store karta hai. Elements memory mein contiguous nahi hotay.

- **Node —** Data + next pointer par mushtamil unit
- **Insert —** Naya node list ke end (ya kisi bhi position) par add karna
- **Traverse —** Head se start karke har node visit karna
- **Delete —** Kisi node ko list se remove karna

Real-Life Example

Train ke compartments jo ek dusray se chained hotay hain — har compartment agle compartment se linked hota hai.

Array vs Linked List

Array ka size fixed hota hai, lekin Linked List runtime mein grow/shrink ho sakti hai.

LINKED LIST

Linked List — C++ Program

```cpp
#include <iostream>
using namespace std;
struct Node { int data; Node* next; };
Node* head = nullptr;

void insert(int val) {
    Node* n = new Node{val, nullptr};
    if (!head) { head = n; return; }
    Node* p = head;
    while (p->next) p = p->next;
    p->next = n;
}

void display() {
    Node* p = head;
    while (p) { cout << p->data << " -> "; p = p->next; }
    cout << "NULL\n";
}

int main() {
    insert(10); insert(20); insert(30);
    display();
    return 0;
}
```

Code Explanation

- struct Node data aur next pointer rakhta hai
- insert() naya node end par jor deta hai
- while (p->next) last node tak jata hai
- display() har node ko print karta hai

Output

```
10 -> 20 -> 30 -> NULL
```

SUMMARY

Queue vs Linked List

Queue

FIFO order — add at rear, remove from front. Fixed-size array ya dynamic structure se implement hoti hai.

Best for: scheduling, buffering, order processing

Linked List

Nodes jo pointers se linked hotay hain. Insertion/deletion array se aasan hai, lekin direct index access nahi hota.

Best for: dynamic-size lists, frequent insert/delete

Dono structures mein data dynamically handle hota hai, lekin access pattern alag hai: Queue sirf front/rear tak access deti hai, jabke Linked List ko traverse karke kisi bhi node tak poncha ja sakta hai.